Core Topics Table

In the following seventeen tables, CS and KA core topics have been listed, one table per knowledge area. For each topic, desired skill levels have been identified and used to estimate the time needed for the instruction of CS Core and KA Core topics. The skill levels should be treated as recommended, not prescriptive. The time needed to cover CS Core and KA Core topics is expressed in terms of **instructional hours**. Instructional hours are hours spent in the classroom imparting knowledge regardless of the pedagogy used. Students are expected to spend additional time after class practicing related skills and exercising professional dispositions.

For convenience, the tables have been listed under three competency areas: Software, Systems, and Applications. The tables on Society, Ethics, and the Profession (SEP) and Mathematical and Statistical Foundations (MSF) are listed last as crosscutting topics that apply to all the competency areas.

Software Competency Area

The core topics in Software Development Fundamentals (SDF) and Algorithmic Foundations (AL) typically constitute the introductory course sequence in computer science and have been listed first.

	Knowledge Area	**Knowledge Units**	**CS Core**	**KA Core**
SDF	Software Development Fundamentals	5	43	
AL	Algorithmic Foundations	5	32	32
FPL	Foundations of Programming Languages	22	21	19
SE	Software Engineering	9	6	21
	Total		102	72

SDF: Software Development Fundamentals

KU	**Topic**	**Skill Level**	**Core**	**Hours**
SDF-Fundamentals	1. Basic concepts such as variables, primitive data types, and expression evaluation 2. How imperative programs work: state and state transitions on execution of statements, flow of control 3. Basic constructs such as assignment statements, conditional and iterative	Develop	CS	18

	statements, basic I/O 4. Key modularity constructs such as functions and related concepts like parameter passing, scope, abstraction, data encapsulation, etc. 5. Input and output using files and APIs 6. Structured data types available in the chosen programming language like sequences, associative containers, others and when and how to use them 7. Libraries and frameworks provided by the language (when/where applicable) 8. Recursion			
SDF-Fundamentals	9. Dealing with runtime errors in programs 10. Basic concepts of programming errors, testing, and debugging 11. Documenting/commenting code at the program and module level	Evaluate Apply	CS	2
SDF-Data-Structures	Standard abstract data types such as lists, stacks, queues, sets, and maps/dictionaries and operations on the data types 4. Strings and string processing	Develop	CS	10
SDF-Data-Structures	Selecting and using appropriate data structures Performance implications of choice of data structure(s)	Evaluate	CS	2
SDF-Algorithms	Concept of algorithm and notion of algorithm efficiency Some common algorithms (e.g., sorting, searching, tree traversal, graph traversal) Impact of algorithms on time/space efficiency of programs	Explain	CS	6
SDF-Practices	Basic testing including test case design Specifying functionality of a module in a natural language	Develop	CS	3
SDF-Practices	3. Programming style that improves readability	Evaluate	CS	1
SDF-Practices	Use of a general-purpose IDE, including its debugger	Apply	CS	1

AL: Algorithmic Foundations

KU	Topic	Skill Level	Core	Hours
AL-Foundational AL-Complexity	2. Arrays 1. Abstract Data Types and Operations 2b i. Foundational Complexity Classes: Constant		CS	1
AL-Foundational AL-Complexity AL-Strategies	11a. Search Algorithms O(n) (e.g., linear array search) 2b iii. Foundational Complexity Classes: Linear 1a. Brute Force	Apply Evaluate Explain	CS	1
AL-Foundational AL-Complexity AL-Strategies	12a. Sorting O(n2), (e.g., selection sort of an array) 2b v. Foundational Complexity classes: Quadratic 1a. Brute Force	Apply Evaluate Explain	CS	1
AL-Foundational AL-Complexity AL-Strategies	11b. Search $O(\log_2 n)$, (e.g., Binary search of an array) 2b ii. Foundational Complexity Classes: *Logarithmic* 1b ii. Decrease-and-Conquer	Apply Evaluate Explain	CS	1
AL-Foundational AL-Complexity AL-Strategies	12b. Sorting $O(n \log n)$, (e.g., Quick, Merge, Tim) 2b iv. Foundational Complexity Classes: *Log Linear* 1c. Divide-and-Conquer	Apply Evaluate Explain	CS	1
AL-Foundational AL-Complexity AL-Strategies	4. Linked Lists 1. Abstract Data Types and Operations 11a. Search $O(n)$, (e.g., linear linked list search) 2b iii. Foundational Complexity Classes: *Linear* 1a. Brute Force	Explain Apply Apply Evaluate Explain	CS	1
AL-Foundational AL-Complexity AL-	5. Stacks 1. Abstract Data Types and Operations 2b i. Foundational complexity classes: *Constant* 6. Queues and Deques 1. Abstract Data Types and Operations	Explain, Apply Explain, Apply	CS	1

Foundational				
AL-Foundational AL-Complexity AL-Strategies	7. Hash Tables/Maps 7a. Collision resolution and complexity 1. Abstract Data Types and Operations 2b i. Foundational complexity classes: *Constant* 1f. Time vs Space tradeoff	Explain Explain Apply Explain Explain	CS	1 1
AL-Foundational AL-Strategies AL-Foundational AL-Strategies	9. Trees 1. Abstract Data Types and Operations 11c. Search Algorithms DFS/BFS 2b. Decrease-and-Conquer 9b. Balanced Trees (e.g., AVL, 2-3, Red-Black, Heap) 1e ii. Transform-and-Conquer: Representation Change (e.g., heapsort)	Explain Apply Apply Explain Apply Explain	CS	1 3
AL-Foundational AL-Foundational AL-Strategies	8. Graphs (e.g., [un]directed, [a]cyclic, [un]connected, [un]weighted) 8a. Representation: Adjacency List vs Matrix 13. Graph Algorithms 13a. Shortest Path (e.g., Dijkstra's, Floyd's) 13b. Minimal, spanning tree (e.g., Prim's, Kruskal's) 1d. Greedy 1e. iv. Dynamic Programming	Explain Apply Apply Explain	CS	1 4
AL-Foundational	1. Abstract Data Types and Operations 3. Records/Structures/Tuples and Objects 10. Sets	Explain	CS	1
AL-Strategies AL-Strategies AL-Strategies AL-Strategies	1. Paradigms (demonstrated in AL-Foundational) 1a. Brute-Force 1b. Decrease-and-Conquer 1c. Divide-and-Conquer 1f. Time-Space Tradeoff 3. Iteration vs Recursion 1e. Transform-and-Conquer 1e i. Instance Simplification (e.g., pre-sorting) 1e iii. Problem Reduction (e.g., least-common-multiple) 2. Handling Exponential Growth (e.g., heuristic A*, backtracking, ranch-and-bound 1e iv. Dynamic Programming	Explain Explain Explain Explain	CS	3 1 1 1

	(e.g., Bellman-Ford, Knapsack, Floyd, Warshall)			
AL-Complexity	1. Complexity Analysis Framework	Explain	CS	1
	2. Asymptotic Complexity Analysis	Explain		
	2a. Big O, Big Omega, and Big Theta			
	2b. Foundational complexity classes demonstrated by AL-Foundational algorithms (with complexity): *Constant, Logarithmic, Linear, Log Linear, Quadratic*, and *Cubic*	Evaluate		1
AL-Complexity	4. Tractability and Intractability	Explain		2
	4a. P, NP, and NP-C complexity classes			
	4b. NP-Complete Problems (e.g., SAT, Knapsack, TSP)			
	4c. Reductions			
AL-Strategies	1a. Paradigms: Exhaustive brute force	Explain		1
	1e. iv. Dynamic Programming			
AL-Complexity	2 vii. Foundational Complexity Classes: *Exponential*	Explain		1
	2b viii. Factorial complexity classes: Factorial *O*(n!) (e.g. All Permutations, Hamiltonian Circuit)			
AL-Models	1. Formal Automata	Explain, Apply	CS	1
	1a. Finite State Automata			
	2a. Regular language, grammar, and expressions			
	1b. Pushdown Automata	Explain, Apply		1
	2b. Context-Free language and grammar			
	1d. Turing Machine	Explain		3
	2d. Recursively Enumerable language and grammar	Explain		
	1c. Linear-Bounded			
	2c. Context-Sensitive language and grammar			1
	2. Formal Languages and Grammars	Explain		2
	4. Decidability, Computability, Halting problem	Explain		
	5. The Church-Turing Thesis	Explain		1
	6. Algorithmic Correctness			
	Invariants (e.g., in iteration, recursion, sorting, heaps)	Explain		
AL-SEP	1. Social, Ethical, and Secure Algorithms	Explain	CS	In SEP Hours
	2. Algorithmic Fairness (e.g., differential privacy)			
	3. Accountability/Transparency			
	4. Responsible algorithms			
	5. Economic and other impacts of algorithms			
	6. Sustainability			

KU	Topic	Skill Level	Core	Hours
FPL-OOP	1. Imperative programming as a subset of object-oriented programming 2. Object-oriented design a. Decomposition into objects carrying state and having behavior b. Class-hierarchy design for modeling 3. Definition of classes: fields, methods, and constructors 4. Subclasses, inheritance (including multiple inheritance), and method overriding 5. Dynamic dispatch – definition of method-call 6. Exception handling 7. Object-oriented idioms for encapsulation a. Privacy, data hiding, and visibility of class members b. Interfaces revealing only method signatures c. Abstract base classes, traits and mixins 8. Dynamic vs static properties 9. Composition vs inheritance 10. Subtyping a. Subtype polymorphism; implicit upcasts in typed languages b. Notion of behavioral replacement – subtypes acting like supertype c. Relationship between subtyping and inheritance	Develop	CS	1. 5
	11. Collection classes, iterators, and other common library components 12. Metaprogramming and reflection	Develop	KA	1
FPL-Functional	1. Lambda expressions and evaluation a. Variable binding and scope rules b. Parameter-passing c. Nested lambda expressions and reduction order 2. Effect-free programming a. Function calls have no side effects, facilitating compositional reasoning. b. Immutable variables and data copying vs reduction	Develop	CS	4

	c. Use of recursion vs loops vs pipelining (map/reduce) 3. Processing structured data (e.g., trees) via functions with cases for each data variant a. Functions defined over compound data in terms of functions applied to the constituent pieces b. Persistent data structures 4. Using higher-order functions (taking, returning, and storing functions)			
	5. Metaprogramming and reflection 6. Function closures (functions using variables in the enclosing lexical environment) a. Basic meaning and definition – creating closures at run-time by capturing the environment b. Canonical idioms: call-backs, arguments to iterators, reusable code via function arguments c. Using a closure to encapsulate data in its environment d. Delayed vs eager evaluation	Explain	KA	3
FPL-Logic	1. Universal vs existential quantifiers 2. First order predicate logic vs higher order logic 3. Expressing complex relations using logical connectives and simpler relations 4. Definitions of Horn clause, facts, goals and subgoals 5. Unification and unification algorithm; unification vs assertion vs expression evaluation 6. Mixing relations with functions 7. Cuts, backtracking, and non-determinism 8. Closed-world vs open-world assumptions	Explain	KA	3
FPL-Scripting	1. Error/exception handling 2. Piping 3. System commands a. Interface with operating systems 4. Environment variables 5. File abstraction and operators 6. Data structures, such as arrays and lists 7. Regular expressions 8. Programs and processes 9. Workflow	Develop	CS	2
FPL-Event-Driven	1. Procedural programming vs reactive programming – advantages of reactive programming in capturing	Develop	CS	2

	events 2. Components of reactive programming – event-source, event signals, listeners and dispatchers, event objects, adapters, event-handlers 3. Stateless and state-transition models of event-based programming 4. Canonical uses such as GUIs, mobile devices, robots, servers			
	5. Using a reactive framework a. Defining event handlers/listeners b. Parameterization of event senders and event arguments c. Externally generated events and program-generated events 6. Separation of model, view, and controller 7. Event-driven and reactive programs as state-transition systems	Develop	KA	2
FPL-Parallel	1. Safety and liveness a. Race conditions. b. Dependencies/preconditions c. Fault models d. Termination 2. Programming models: One or more of the following a. Actor models b. Procedural and reactive models c. Synchronous/asynchronous programming models d. Data parallelism 3. Properties a. Order-based properties i. Commutativity ii. Independence b. Consistency-based properties i. Atomicity ii. Consensus 4. Execution control: a. Async await b. Promises c. Threads 5. Communication and coordination a. Mutexes b. Message-passing	Develop	CS	3

	c. Shared memory d. Cobegin-coend e. Monitors f. Channels g. Threads h. Guards			
	6. Futures 7. Language support for data parallelism such as forall, loop unrolling, map/reduce 8. Effect of memory-consistency models on language semantics and correct code generation 9. Representational State Transfer Application Programming Interfaces (REST APIs) 10. Technologies and approaches: cloud computing, high performance computing, quantum computing, ubiquitous computing 11. Overheads of message-passing 12. Granularity of program for efficient exploitation of concurrency 13. Concurrency and other programming paradigms (e.g., functional)	Explain	KA	2
FPL-Types	1. A type as a set of values together with a set of operations a. Primitive types (e.g., numbers, Booleans) b. Compound types built from other types (e.g., records/structs, unions, arrays, lists, functions, references using set operations) 2. Association of types to variables, arguments, results, and fields 3. Type safety as an aspect of program correctness 4. Type safety and errors caused by using values inconsistently given their intended types 5. Goals and limitations of static and dynamic typing a. Detecting and eliminating errors as early as possible 6. Generic types (parametric polymorphism) a. Definition and advantages of polymorphism – parametric, subtyping, overloading, and coercion b. Comparison of monomorphic and polymorphic types c. Comparison with ad-hoc polymorphism (overloading) and subtype polymorphism	Develop	CS	3

	d. Generic parameters and typing e. Use of generic libraries such as collections f. Comparison with ad hoc polymorphism (overloading) and subtype polymorphism g. Prescriptive vs descriptive polymorphism h. Implementation models of polymorphic types i. Subtyping			
	7. Type equivalence – structural vs name equivalence 8. Complementary benefits of static and dynamic typing a. Errors early vs errors late/avoided b. Enforce invariants during code development and code maintenance vs postpone typing decisions while prototyping and conveniently allow flexible coding patterns such as heterogeneous collections c. Typing rules i. Rules for function, product, and sum types d. Avoid misuse of code vs allow more code reuse e. Detect incomplete programs vs allow incomplete programs to run f. Relationship to static analysis g. Decidability	Develop	KA	4
FPL-Systems	1. Data structures for translation, execution, translation, and code mobility such as stack, heap, aliasing (sharing using pointers), indexed sequence and string 2. Direct, indirect, and indexed access to memory location 3. Run-time representation of data abstractions such as variables, arrays, vectors, records, pointer-based data elements such as linked-lists and trees, and objects 4. Abstract low-level machine with simple instruction, stack, and heap to explain translation and execution 5. Run-time layout of memory: activation record (with various pointers), static data, call-stack, heap a. Translating selection and iterative constructs to control-flow diagrams b. Translating control-flow diagrams to low level abstract code c. Implementing loops, recursion, and tail calls d. Translating function/procedure calls and return	Develop	CS	3

	from calls, including different parameter-passing mechanisms using an abstract machine 6. Memory management a. Low level allocation and accessing of high-level data structures such as basic data types, n-dimensional array, vector, record, and objects b. Return from procedure as automatic deallocation mechanism for local data elements in the stack c. Manual memory management: allocating, de-allocating, and reusing heap memory d. Automated memory management – garbage collection as an automated technique using the notion of reachability 7. Green computing			
FPL-Translation	1. Execution models for JIT (Just-In-Time), compiler, interpreter 2. Use of intermediate code, e.g., bytecode. 3. Limitations and benefits of JIT, compiler, and interpreter 4. Cross compilers/transpilers 5. BNF and extended BNF representation of context-free grammar 6. Parse tree using a simple sentence such as arithmetic expression or if-then-else statement 7. Execution as native code or within a virtual machine 8. Language translation pipeline – syntax analysis, parsing, optional type-checking, translation/code generation and optimization, linking, loading, execution	Explain	CS	2
	9. Run-time representation of core language constructs such as objects (method tables) and functions that can be passed as parameters to and returned from functions (closures) 10. Secure compiler development	Explain	KA	2
FPL-Abstraction	1. BNF and regular expressions 2. Programs that take (other) programs as input such as interpreters, compilers, type-checkers, documentation generators 3. Components of a language a. Definitions of alphabets, delimiters, sentences,	Explain	KA	3

	syntax, and semantics b. Syntax vs semantics 4. Program as a set of non-ambiguous meaningful sentences 5. Basic programming abstractions – constants, variables, declarations (including nested declarations), command, expression, assignment, selection, definite and indefinite iteration, iterators, function, procedure, modules, exception handling 6. Mutable vs immutable variables: advantages and disadvantages of reusing existing memory location vs advantages of copying and keeping old values; storing partial computation vs recomputation 7. Types of variables – static, local, nonlocal, global; need and issues with nonlocal and global variables 8. Scope rules – static vs dynamic; visibility of variables; side-effects 9. Side-effects induced by nonlocal variables, global variables, and aliased variables			

SE: Software Engineering

KU	Topic	Skill Level	Core	Hours
SE-Teamwork	1. Effective communication 2. Common causes of team conflict, and approaches for conflict resolution 3. Cooperative programming 4. Roles and responsibilities in a software team 5. Team processes 6. Importance of team diversity and inclusion	Evaluate	CS	2
SE-Teamwork	7. Interfacing with stakeholders, as a team 8. Risks associated with physical, distributed, hybrid and virtual teams	Explain	KA	2
SE-Tools	1. Software configuration management and version control	Evaluate	CS	1
SE-Tools	2. Release management 3. Testing tools including static and dynamic analysis tools			

	4. Software process automation 5. Design and communication tools (docs, diagrams, common forms of design diagrams like UML) 6. Tool integration concepts and mechanisms 7. Use of modern IDE facilities – debugging, refactoring, searching/indexing, etc.	Explain	KA	3
SE-Requirements	1. Describe functional requirements using, for example, use cases or user stories 2. Properties of requirements including consistency, validity, completeness, and feasibility 3. Requirements elicitation 4. Non-functional requirements, for example, security, usability, or performance (aka Quality Attributes) 5. Risk identification and management 6. Communicating and/or formalizing requirement specifications	Apply	KA	2
SE-Design	1. System design principles 2. Software architecture 3. Programming in the large vs programming in the small 4. Code smells and other indications of code quality, distinct from correctness.	Explain	CS	1
SE-Design	5. API design principles 6. Identifying and codifying data invariants and time invariants 7. Structural and behavioral models of software designs 8. Data design 9. Requirement traceability	Apply	KA	4
SE-Construction	1. Practical small-scale testing 2. Documentation	Apply	CS	1
SE-Construction	3. Coding Style 4. “Best Practices” for coding 5. Debugging 6. Logging 7. Use of libraries and frameworks developed by others	Apply	KA	3
SE-Validation	1. Verification and validation concepts 2. Why testing matters	Explain	CS	1

	3. Testing objectives 4. Test kinds 5. Stylistic differences between tests and production code			
SE-Validation	6. Test planning and generation 7. Test development (see SDF) 8. Verification and validation in the development cycle 9. Domain specific verification and validation challenges	Explain	KA	4
SE-Refactoring	1. Hyrum's Law 2. Backward Compatibility 3. Refactoring 4. Versioning	Explain	KA	1
SE-Reliability	1. Concept of reliability 2. Identifying reliability requirements (see SEP) 3. Software failures vs defect injection/detection 4. Software reliability, system reliability and failure behavior (cross-reference SF/Reliability Through Redundancy) 5. Defect injection and removal cycle, and different approaches for defect removal 6. Compare the "error budget" approach to reliability with the "error-free" approach, and identify domains where each is relevant	Explain	KA	4

Systems Competency Area

The core topics in Architecture and Organization (AR) and Operating Systems (OS) are typically covered early in the curriculum and have been listed first. Data Management (DM) and Security (SEC) topics listed in this section can be applied to all three competency areas.

	Knowledge Area	**Knowledge Units**	**CS Core**	**KA Core**
AR	Architecture and Organization	9	9	16
OS	Operating Systems	14	8	13
NC	Networking and Communication	8	7	24

PDC	Parallel and Distributed Computing	5	9	26
SF	Systems Fundamentals	8	18	8
DM	Data Management	12	10	26
SEC	Security	6	6	35
	Total		67	148

AR: Architecture and Organization

KU	Topic	Skill Level	Core	Hours
AR-Logic	1. Combinational vs sequential logic/field programmable gate arrays (FPGAs) a. Fundamental combinational b. Sequential logic building block	Explain	KA	3
	2. Computer-aided design tools that process hardware and architectural representations 3. High-level synthesis a. Register transfer notation b. Hardware description language (e.g., Verilog/VHDL/Chisel) 4. System-on-chip (SoC) design flow 5. Physical constraints a. Gate delays b. Fan-in and fan-out c. Energy/power d. Speed of light	Evaluate		
AR-Representation	1. Overview and history of computer architecture 2. Bits, bytes, and words 3. Unsigned, signed and two's complement representations 4. Numeric data representation and number bases a. Fixed-point b. Floating-point 5. Representation of non-numeric data 6. Representation of records, arrays and UTF data types	Apply	CS	1

AR-Assembly	1. von Neumann machine architecture 2. Control unit: instruction fetch, decode, and execution 3. Introduction to SIMD vs MIMD and the Flynn taxonomy 4. Shared memory multiprocessors/multicore organization	Explain	CS	1
AR-Assembly	5. Instruction set architecture (ISA) (e.g., x86, ARM, and RISC-V) a. Instruction formats b. Data manipulation, control, I/O c. Addressing modes d. Machine language programming e. Assembly language programming 6. Subroutine call and return mechanisms 7. I/O and interrupts 8. Heap, static, stack, and code segments	Develop	KA	2
AR-Memory	1. Memory hierarchy: the importance of temporal and spatial locality 2. Main memory organization and operations 3. Persistent memory (e.g., SSD, standard disks) 4. Latency, cycle time, bandwidth, and interleaving 7. Virtual memory (hardware support) 8. Fault handling and reliability 9. Reliability a. Error coding b. Data compression c. Data integrity 10. In-Memory Processing (PIM)	Explain	CS	6
	5. Cache memories a. Address mapping b. Block size c. Replacement and store policy 6. Multiprocessor cache coherence	Evaluate		
	1. I/O fundamentals a. Handshaking and buffering b. Programmed I/O c. Interrupt-driven I/O 2. Interrupt structures: vectored and prioritized, interrupt acknowledgment	Explain	CS	1

AR-IO	3. I/O devices (e.g., mouse, keyboard, display, camera, sensors, actuators) 4. External storage, physical organization, and drives 5. Bus fundamentals a. Bus protocols b. Arbitration c. Direct-memory access (DMA)			
AR-Organization	1. Implementation of simple datapaths, including instruction pipelining, hazard detection, and resolution 2. Control unit a. Hardwired implementation b. Microprogrammed realization	Develop	KA	2
	3. Instruction pipelining 4. Introduction to instruction-level parallelism (ILP)	Explain		
AR-Performance-Energy	1. Performance-energy evaluation (introduction): performance, power consumption, memory, and communication costs 2. Branch prediction, speculative execution, out-of-order execution, Tomasulo's algorithm	Evaluate	KA	2
AR-Performance-Energy	3. Enhancements for vector processors and GPUs 4. Hardware support for multithreading a. Race conditions b. Lock implementations c. Point-to-point synchronization d. Barrier implementation 5. Scalability 6. Alternative architectures, such as VLIW/EPIC, accelerators, and other special-purpose processors 7. Dynamic voltage and frequency scaling (DVFS) 8. Dark Silicon	Explain	KA	1
AR-Heterogeneity	1. SIMD and MIMD architectures (e.g., General-Purpose GPUs, TPUs, and NPUs) 2. Heterogeneous memory system a. Shared memory versus distributed memory b. Volatile vs non-volatile memory c. Coherence protocols 3. Domain-Specific Architectures (DSAs) a. Machine Learning Accelerator	Explain	KA	2

	b. In-networking computing c. Embedded systems for emerging applications d. Neuromorphic computing e. Edge computing devices 4. Packaging and integration solutions such as 3DIC and chiplets 5. Machine learning in architecture design a. AI algorithms for workload analysis b. Optimization of architecture configurations for performance and power efficiency			
AR-Security	1. Principles of Secure Hardware a. Security Risk Analysis, Asset Protection, and Threat Model b. Cryptographic Acceleration with Hardware c. Support for virtualization (e.g., OS isolation) 2. Roots of trust in hardware, Physically Unclonable Functions (PUF) 3. Hardware Random Number Generators 4. Memory protection extensions a. Runtime pointer bounds checking (e.g., buffer overflow) b. Protection at the microarchitectural level c. Protection at the ISA level 5. Trusted Execution Environment (TEE) a. Trusted Computer Base Protections b. Protecting virtual machines c. Protecting containers d. Trusted software modules (Enclaves)	Explain	KA	2
AR-Quantum	1. Principles a. The wave-particle duality principle b. The uncertainty principle in the double-slit experiment c. What is a Qubit? Superposition and measurement; Photons as qubits d. Systems of two qubits; Entanglement; Bell states; The No-Signaling theorem 2. Axioms of QM: superposition principle, measurement axiom, unitary evolution 3. Single qubit gates for the circuit model of quantum computation: X, Z, H.	Explain	KA	2

	4. Two qubit gates and tensor products; Working with matrices 5. The No-Cloning Theorem; The Quantum Teleportation protocol 6. Algorithms a. Simple quantum algorithms (Bernstein-Vazirani, Simon's algorithm) b. Implementing Deutsch-Josza with Mach-Zehnder Interferometers c. Quantum factoring (Shor's Algorithm) d. Quantum search (Grover's Algorithm) 7. Implementation aspects a. The physical implementation of qubits b. Classical control of a Quantum Processing Unit (QPU) c. Error mitigation and control. NISQ and beyond. 8. Emerging Applications a. Post-quantum encryption b. The Quantum Internet c. Adiabatic quantum computation (AQC) and quantum annealing			

OS: Operating Systems

KU	Topic	Skill Level	Core	Hours
OS-Purpose	1. Operating system as mediator between general purpose hardware and application-specific software 2. Universal operating system functions 3. Extended and/or specialized operating system functions 4. Design issues 5. Influences of security, networking, multimedia, parallel and distributed computing 6. Overarching concern of security/protection: Neglecting to consider security at every layer creates an opportunity to inappropriately access resources.	Explain	CS	2
OS-Principles	1. Operating system software design and approaches 2. Abstractions, processes, and resources	Explain	CS	2

	3. Concept of system calls and links to application program interfaces 4. The evolution of the link between hardware architecture and the operating system functions 5. Protection of resources means protecting some machine instructions/functions 6. Leveraging interrupts from hardware level: service routines and implementations 7. Concept of user/system state and protection, transition to kernel mode using system calls 8. Mechanism for invoking of system calls, the corresponding mode and context switch and return from interrupt 9. Performance costs of context switches and associated cache flushes when performing process switches in Spectre-mitigated environments			
OS-Concurrency	1. Thread abstraction relative to concurrency 2. Race conditions, critical regions (role of interrupts if needed) 3. Deadlocks and starvation 4. Multiprocessor issues (spin-locks, reentrancy) 5. Multiprocess concurrency vs multithreading	Explain	CS	2
	6. Thread creation, states, structures 7. Thread APIs 8. Deadlocks and starvation (necessary conditions/mitigations) 9. Implementing thread safe code (semaphores, mutex locks, cond vars) 10. Race conditions in shared memory	Apply	KA	1
OS-Protection	1. Overview of operating system security mechanisms 2. Attacks and antagonism (scheduling, etc.) 3. Review of major vulnerabilities in real operating systems 4. Operating systems mitigation strategies such as backups	Apply	CS	2
	5. Policy/mechanism separation 6. Security methods and devices 7. Protection, access control, and authentication	Apply	KA	1
OS-Scheduling	1. Preemptive and non-preemptive scheduling 2. Schedulers and policies.	Explain	KA	2

	3. Concepts of Symmetric Multi-Processor (SMP) multiprocessor scheduling and cache coherence 4. Timers (e.g., building many timers out of finite hardware timers) 5. Fairness and starvation			
OS-Process	1. Processes and threads relative to virtualization – Protected memory, process state, memory isolation, etc. 2. Memory footprint/segmentation (stack, heap, etc.) 3. Creating and loading executables, shared libraries, and dynamic linking 4. Dispatching and context switching 5. Interprocess communication	Explain	KA	2
OS-Memory	1. Review of physical memory, address translation and memory management hardware 2. Impact of memory hierarchy including cache concept, cache lookup, etc. on operating system mechanisms and policy 3. Logical and physical addressing, address space virtualization 4. Concepts of paging, page replacement, thrashing and allocation of pages and frames 5. Allocation/deallocation/storage techniques (algorithms and data structure) performance and flexibility 6. Memory caching and cache coherence and the effect of flushing the cache to avoid speculative execution vulnerabilities 7. Security mechanisms and concepts in memory management including sandboxing, protection, isolation, and relevant vectors of attack	Explain	KA	2
OS-Devices	1. Buffering strategies 2. Direct Memory Access and Polled I/O, Memory-mapped I/O 3. Historical and contextual - Persistent storage device management (magnetic, SSD, etc.)	Explain	KA	1
OS-Files	1. Concept of a file including Data, Metadata, Operations and Access-mode 2. File system mounting 3. File access control 4. File sharing	Explain	KA	2

	5. Basic file allocation methods including linked, allocation table, etc. 6. File system structures comprising file allocation including various directory structures and methods for uniquely identifying files (name, identified or metadata storage location) 7. Allocation/deallocation/storage techniques (algorithms and data structure) impact on performance and flexibility (i.e. Internal and external fragmentation and compaction) 8. Free space management such as using bit tables vs linking 9. Implementation of directories to segment and track file location			
OS-Advanced-Files	1. File systems: partitioning, mount/unmount, virtual file systems 2. In-depth implementation techniques 3. Memory-mapped files 4. Special-purpose file systems 5. Naming, searching, access, backups 6. Journaling and log-structured file systems	Explain	KA	1
OS-Virtualization	1. Using virtualization and isolation to achieve protection and predictable performance 2. Advanced paging and virtual memory 3. Virtual file systems and virtual devices 4. Containers and their comparison to virtual machine 5. Thrashing	Explain	KA	1
OS-Real-time	1. Process and task scheduling 2. Deadlines and real-time issues 3. Low-latency vs ”soft real-time" vs "hard real time"	Explain	KA	1
OS-Faults	1. Reliable and available systems 2. Software and hardware approaches to address tolerance (RAID)	Explain	KA	1
OS-SEP	1. Open source in operating systems 2. End-of-life issues with sunsetting operating systems	Explain	KA	-

KU	Topic	Skill Level	Core	Hours
NC-Fundamentals	1. Importance of networking in contemporary computing, and associated challenges	Explain	CS	3
	2. Organization of the Internet a. Users b. Internet Service Providers c. Autonomous systems d. Content providers e. Content delivery networks	Explain		
	3. Switching techniques a. Circuit Switching b. Packet Switching	Evaluate		
	4. Layers and their roles a. Application b. Transport c. Network d. Datalink e. Physical	Explain		
	5. Layering principles a. Encapsulation b. Hourglass model	Explain		
	6. Network elements a. Routers b. Switches c. Hubs d. Access points e. Hosts	Explain		
	7. Basic queueing concepts a. Relationship with latency b. Relationship with Congestion c. Relationship with Service levels	Explain		
NC-Applications	1. Naming and address schemes. a. DNS b. IP addresses	Explain	CS	4

	c. Uniform Resource Identifiers			
	2. Distributed application paradigms a. Client/server b. Peer-to-peer c. Cloud d. Edge e. Fog	Evaluate		
	3. Diversity of networked application demands a. Latency b. Bandwidth c. Loss tolerance	Explain		
	4. Application-layer development using one or more protocols: a. HTTP b. SMTP c. POP3	Develop		
	5. Interactions with TCP, UDP, and Socket APIs.	Explain		
NC-Reliability	1. Unreliable delivery a. UDP b. Other	Explain	KA	6
	2. Principles of reliability a. Delivery without loss b. Duplication c. Out of order	Develop		
	3. Error control a. Retransmission b. Error correction	Evaluate		
	4. Flow control a. Stop and wait b. Window based	Develop		
	5. Congestion control a. Implicit congestion notification b. Explicit congestion notification	Explain		
	6. TCP and performance issues a. Tahoe b. Reno	Evaluate		

	c. Vegas d. Cubic e. QUIC			
NC-Routing	1. Routing paradigms and hierarchy a. Intra/inter domain b. Centralized and decentralized c. Source routing d. Virtual circuits e. QoS	Evaluate	KA	4
	2. Forwarding methods a. Forwarding tables b. Matching algorithms	Apply		
	3. IP and Scalability issues a. NAT b. CIDR c. BGP d. Different versions of IP	Explain		
NC-SingleHop	1. Introduction to modulation, bandwidth, and communication media	Explain	KA	3
	2. Encoding and Framing.	Evaluate		
	3. Medium Access Control (MAC) a. Random access b. Scheduled access	Evaluate		
	4. Ethernet	Explain		
	5. Switching	Apply		
	6. Local Area Network Topologies (e.g. data center networks)	Explain		
NC-Security	1. General intro about security a. Threats b. Vulnerabilities c. Countermeasures	Explain	KA	4
	2. Network specific threats and attack types a. Denial of service b. Spoofing c. Sniffing	Explain		

	d. Traffic redirection e. Attacker-in-the-middle f. Message integrity attacks g. Routing attacks h. Traffic analysis			
	3. Countermeasures [Shared with Security] a. Cryptography (e.g., SSL, symmetric/asymmetric). b. Architectures for secure networks (e.g., secure channels, secure routing protocols, secure DNS, VPNs, DMZ, Zero Trust Network Access, hyper network security, anonymous communication protocols, isolation) c. Network monitoring, intrusion detection, firewalls, spoofing and DoS protection, honeypots, tracebacks, BGP Sec.	Explain		
NC-Mobility	1. Principles of cellular communication (e.g., 4G, 5G)	Explain	KA	3
	2. Principles of Wireless LANs (mainly 802.11)	Explain		
	3. Device to device communication	Explain		
	4. Multihop wireless networks	Explain		
	5. Examples (e.g., ad hoc networks, opportunistic, delay tolerant)	Explain		
NC-Emerging	1. Middleboxes (e.g., filtering, deep packet inspection, load balancing, NAT, CDN)	Explain	KA	4
	2. Virtualization (e.g. SDN, Data Center Networks)	Explain		
	3. Quantum Networking (e.g. Intro to the domain, teleportation, security, Quantum Internet)	Explain		

PDC: Parallel and Distributed Computing

KU	Topic	Skill Level	Core	Hours

PDC-Programs	1. Parallelism a. Declarative parallelism – determining which actions may, or must not, be performed in parallel, at the level of instructions, functions, closures, composite actions, sessions, tasks, and services is the main idea underlying PDC algorithms; failing to do so is the main source of errors. b. Defining order – for example, using happens-before relations or series/parallel directed acyclic graphs representing programs c. Independence – determining when ordering doesn't matter, in terms of commutativity, dependencies, preconditions d. Ensuring ordering among otherwise parallel actions when necessary, including locking, safe publication; and imposing communication – sending a message happens before receiving it; conversely relaxing when unnecessary 2. Distribution a. Defining places as devices executing actions, including hardware components, remote hosts, may also include external, uncontrolled devices, hosts, and users b. One device may time-slice or otherwise emulate multiple parallel actions by fewer processors by scheduling and virtualization. c. Naming or identifying places (e.g., device IDs) and actions as parties (e.g., thread IDs) d. Activities across places may communicate across media 3. Starting activities a. Options that enable actions to be performed (eventually) at places range from hardwiring to configuration scripts; also establishing communication and resource management; these are expressed differently across languages and contexts, usually relying on automated provisioning and management by platforms b. Procedural: Enabling multiple actions to start at a given program point; for example, starting	Explain	CS	2

	new threads, possibly scoping, or otherwise organizing them in hierarchical groups c. Reactive: Enabling upon an event by installing an event handler, with less control of when actions begin or end d. Dependent: Enabling upon completion of others; for example, sequencing sets of parallel actions e. Granularity: Execution cost of action bodies should outweigh the overhead of arranging. 4. Execution Properties a. Nondeterministic execution of unordered actions b. Consistency – ensuring agreement among parties about values and predicates when necessary to avoid races, maintain safety and atomicity, or arrive at consensus. c. Fault tolerance – handling failures in parties or communication, including (Byzantine) misbehavior due to untrusted parties and protocols, when necessary to maintain progress or availability. d. Tradeoffs are one focus of evaluation			
	5. One or more of the following mappings and mechanisms across layered systems a. CPU data- and instruction-level parallelism. b. SIMD and heterogeneous data parallelism c. Multicore scheduled concurrency, tasks, actors d. Clusters, clouds; elastic provisioning e. Networked distributed systems f. Emerging technologies such as quantum computing and molecular computing	Explain, Develop	KA	2
PDC-Communication	1. Media a. Varieties – channels (message passing or IO), shared memory, heterogeneous, data stores b. Reliance on the availability and nature of underlying hardware, connectivity, and protocols; language support, emulation 2. Channels a. Explicit (usually named) party-to-party communication media	Explain	CS	2

	b. APIs: sockets, architectural and language-based constructs, and layered constructs such as RPC (remote procedure call) c. IO channel APIs 3. Memory a. Shared memory architectures in which parties directly communicate only with memory at given addresses, with extensions to heterogeneous memory supporting multiple memory stores with explicit data transfer across them; for example, GPU local and shared memory, Direct Memory Access (DMA) b. Memory hierarchies – multiple layers of sharing domains, scopes, and caches; locality: latency, false-sharing c. Consistency properties – bitwise atomicity limits, coherence, local ordering 4. Data Stores a. Cooperatively maintained data structures implementing maps and related ADTs. b. Varieties – owned, shared, sharded, replicated, immutable, versioned			
	5. One or more of the following properties and extensions a. Topologies – unicast, multicast, mailboxes, switches; routing via hardware and software interconnection networks b. Media concurrency properties – ordering, consistency, idempotency, overlapping communication with computation c. Media performance – latency, bandwidth (throughput) contention (congestion), responsiveness (liveness), reliability (error and drop rates), protocol-based progress (acks, timeouts, mediation) d. Media security properties – integrity, privacy, authentication, authorization e. Data formats – marshaling, validation, encryption, compression f. Channel policies: Endpoints, Sessions, buffering, saturation response (waiting vs dropping), Rate control	Explain, Develop	KA	6

	g. Multiplexing and demultiplexing many relatively slow I/O devices or parties; completion-based and scheduler-based techniques; async-await, select and polling APIs h. Formalization and analysis of channel communication; for example, CSP i. Applications of queuing theory to model and predict performance j. Memory models – sequential and release/acquire consistency k. Memory management, including reclamation of shared data; reference counts and alternatives l. Bulk data placement and transfer; reducing message traffic and improving locality; overlapping data transfer and computation; impact of data layout such as array-of-structs vs struct-of-arrays m. Emulating shared memory: distributed shared memory, Remote Direct Memory Access (RDMA) n. Data store consistency – atomicity, linearizability, transactionality, coherence, causal ordering, conflict resolution, eventual consistency, blockchains o. Faults, partitioning, and partial failures; voting; protocols such as Paxos and Raft p. Design tradeoffs among consistency, availability, partition (fault) tolerance; impossibility of meeting all at once q. Security and trust: Byzantine failures, proof of work and alternatives			
PDC-Coordination	1. Dependencies a. Initiation or progress of one activity may be dependent on other activities, so as to avoid race conditions, ensure termination, or meet other requirements. b. Ensuring progress by avoiding dependency cycles, using monotonic conditions, removing inessential dependencies 2. Control constructs and design patterns a. Completion-based – barriers, joins, including termination control	Explain	CS	2

	b. Data-enabled – queues, producer-consumer designs c. Condition-based – polling, retrying, backoffs, helping, suspension, signaling, timeouts d. Reactive: enabling and triggering continuations 3. Atomicity a. Atomic instructions enforced local access orderings b. Locks and mutual exclusion; lock granularity c. Deadlock avoidance – ordering, coarsening, randomized retries; encapsulation via lock managers d. Common errors: Failing to lock or unlock when necessary, holding locks while invoking unknown operations. e. Avoiding locks – replication, read-only, ownership, and non-blocking constructions			
	4. One or more of the following properties and extensions a. Progress properties including lock-free, wait-free, fairness, priority scheduling, interactions with consistency, reliability b. Performance with respect to contention, granularity, convoying, scaling c. Non-blocking data structures and algorithms d. Ownership and resource control e. Lock variants and alternatives: sequence locks, read-write locks; Read-Copy-Update (RCU), reentrancy; tickets; controlling spinning versus blocking f. Transaction-based control – optimistic and conservative g. Distributed locking: reliability h. Alternatives to barriers: clocks; counters, virtual clocks; dataflow and continuations; futures and RPC; consensus-based, gathering results with reducers and collectors i. Speculation, selection, cancellation; observability and security consequences j. Resource control using semaphores and condition variables	Explain, Develop	KA	6

	k. Control flow – scheduling computations, series-parallel loops with (possibly elected) leaders, pipelines and streams, nested parallelism. l. Exceptions and failures, handlers, detection, timeouts, fault tolerance, voting			
PDC-Evaluation:	1. Safety and liveness requirements in terms of temporal logic constructs to express “always” and “eventually” 2. Identifying, testing for, and repairing violations, including common forms of errors such as failure to ensure necessary ordering (race errors), atomicity (including check-then-act errors), and termination (livelock) 3. Performance requirements metrics for throughput, responsiveness, latency, availability, energy consumption, scalability, resource usage, communication costs, waiting and rate control, fairness; service level agreements 4. Performance impact of design and implementation choices, including granularity, overhead, and energy consumption 5. Estimating scalability limitations, for example, using Amdahl’s Law or Universal Scalability Law	Explain, Evaluate	CS	1
	6. One or more of the following methods and tools a. Extensions to formal sequential requirements such as linearizability b. Protocol, session, and transactional specifications c. Use of tools such as Unified Modelling Language (UML), Temporal Logic of Actions (TLA), program logics d. Security analysis: safety and liveness in the presence of hostile or buggy behaviors by other parties; required properties of communication mechanisms (for example lack of cross-layer leakage), input screening, rate limiting e. Static analysis applied to correctness, throughput, latency, resources, energy f. Directed Acyclic Graph (DAG) model analysis of algorithmic efficiency (work, span, critical paths)	Explain, Evaluate	KA	3

	g. Testing and debugging; tools such as race detectors, fuzzers, lock dependency checkers, unit/stress/torture tests, visualizations, continuous integration, continuous deployment, and test generators h. Measuring and comparing throughput, overhead, waiting, contention, communication, data movement, locality, resource usage, behavior in the presence of excessive numbers of events, clients, or threads i. Application domain specific analyses and evaluation techniques			
PDC-Algorithms	1. Expressing and implementing algorithms in given languages and frameworks, to initiate activities (for example threads), use shared memory constructs, and channel, socket, and/or remote procedure call APIs a. Data parallel examples including map/reduce b. Using channel, socket, and/or RPC APIs in a specified language, with program control for sending (usually procedural) vs receiving (usually reactive or RPC-based) c. Using locks, barriers, and/or synchronizers to maintain liveness without introducing races 2. Survey of common application domains across multicore, reactive, data parallel, cluster, cloud, open distributed systems, and frameworks	Explain, Develop	CS	2
	3. One or more of the following algorithmic domains a. Linear algebra – vector and matrix operations, numerical precision/stability, applications in data analytics and machine learning b. Data processing – sorting, searching and retrieval, concurrent data structures c. Graphs, search, and combinatorics – marking, edge-parallelization, bounding, speculation, network-based analytics d. Modeling and simulation – differential equations; randomization, N-body problems, genetic algorithms e. Computational Logic – satisfiability (SAT), concurrent logic programming f. Graphics and computational geometry – transforms, rendering, ray-tracing	Explain, Develop, Evaluate	KA	9

	g. Resource management – allocating, placing, recycling and scheduling processors, memory, channels, and hosts; exclusive vs shared resources; static, dynamic, and elastic algorithms; real-time constraints; batching, prioritization, partitioning; decentralization via work-stealing and related techniques h. Services – implementing web APIs, electronic currency, transaction systems, multiplayer games			

SF: Systems Fundamentals

KU	Topic	Skill Level	Core	Hours
SF-Overview	1. Basic building blocks and components of a computer (gates, flip-flops, registers, interconnections; datapath + control + memory) 2. Hardware as a computational paradigm – fundamental logic building blocks; logic expressions, minimization, sum of product forms 3. Programming abstractions, interfaces, and use of libraries 4. Distinction and interaction between application and OS services, remote procedure call 5. Basic concept of pipelining, overlapped processing stages 6. Basic concept of scaling: going faster vs handling larger problems	Explain	CS	3
SF-Foundations	1. Digital vs Analog/Discrete vs Continuous Systems 2. Simple logic gates, logical expressions, Boolean logic simplification 3. Clocks, State, Sequencing 4. State and state transition (e.g., starting state, final state, life cycle of states) 5. Finite state machines (e.g., NFA, DFA) 6. Combinational Logic, Sequential Logic, Registers, Memories 7. Computers and Network Protocols as examples of State Machines	Apply	CS	4

	8. Sequential vs parallel processing 9. Application-level sequential processing – single thread 10. Simple application-level parallel processing – request level (web services/client-server/distributed), single thread per server, multiple threads with multiple servers, pipelining			
SF-Resource	1. Different types of resources (e.g., processor share, memory, disk, net bandwidth) 2. Common resource allocation/scheduling algorithms (e.g., first-come-first-serve, priority-based scheduling, fair scheduling, and preemptive scheduling)	Explain	CS	1
	3. Advantages and disadvantages of common scheduling algorithms	Explain	KA	1
SF-Performance	1. Latencies in computer systems a. Speed of light and computers (one foot per nanosecond vs one GHz clocks) b. Memory vs disk latencies vs across-the-network memory 2. Caches and the effects of spatial and temporal locality on performance in processors and systems 3. Caches and cache coherency in databases, operating systems, distributed systems, and computer architecture 4. Introduction to the processor memory hierarchy	Apply	CS	2
	5. The formula for average memory access time 6. Rationale of virtualization and isolation – protection and predictable performance 7. Levels of indirection, illustrated by virtual memory for managing physical memory resources 8. Methods for implementing virtual memory and virtual machines	Apply	KA	2
SF-Evaluation	1. Performance figures of merit 2. Workloads and representative benchmarks, and methods of collecting and analyzing performance figures of merit 3. CPI (Cycles per Instruction) equation as a tool for understanding tradeoffs in the design of instruction	Evaluate	CS	2

	sets, processor pipelines, and memory system organizations 4. Amdahl's Law: the part of the computation that cannot be sped up limits the effect of the parts that can be 5. Order of magnitude analysis (Big O notation) 6. Analysis of slow and fast paths of a system 7. Events on their effect on performance (e.g., instruction stalls, cache misses, page faults)			
	8. Analytical tools to guide quantitative evaluation 9. Understanding layered systems, workloads, and platforms, their implications for performance, and the challenges they represent for evaluation 10. Microbenchmarking pitfalls	Evaluate	KA	2
SF-Reliability	1. Distinction between bugs and faults 2. Reliability vs availability 3. Reliability through redundancy a. check and retry b. redundant encoding (error correction codes, CRC, FEC) c. duplication/mirroring/replicas	Evaluate	CS	2
	4. Other approaches to reliability (e.g., journaling)	Evaluate	KA	1
SF-Security	1. Common system security issues (e.g., viruses, denial-of-service attacks, and eavesdropping) 2. Countermeasures a. Cryptography b. Security architecture	Evaluate	CS	2
	3. Representative countermeasure systems a. Intrusion detection systems, firewalls	Evaluate	KA	1
SF-Design	1. Common criteria of system design (e.g., liveness, safety, robustness, scalability, and security)	Design	CS	2
	2. Designs of representative systems (e.g., Apache web server, Spark, and Linux)	Design	KA	1

DM: Data Management

KU	Topic	Skill Level	Core	Hours

DM-Data	1. The Data Life Cycle	Evaluate	CS	2
DM-Core	1. Purpose and advantages of database systems 2. Components of database systems 3. Design of core DBMS functions (e.g., query mechanisms, transaction management, buffer management, access methods) 4. Database architecture, data independence, and data abstraction 5. Transaction management 6. Normalization 7. Approaches for managing large volumes of data (e.g., NoSQL database systems, use of MapReduce) 8. How to support CRUD-only applications 9. Distributed databases/cloud-based systems 10. Structured, semi-structured, and unstructured databases	Explain	CS	2
	11. Use of a declarative query language	Develop		
DM-Core	12. Systems supporting structured and/or stream content	Explain	KA	1
DM-Modeling	1. Data modeling 2. Relational data models	Develop	CS	2
DM-Modeling	3. Conceptual models (e.g., entity-relationship, UML diagrams) 4. Semi-structured data model (expressed using DTD, XML, or JSON Schema, for example)	Explain	KA	3
DM-Relational	1. Entity and referential integrity a. Candidate key, superkeys 2. Relational database design	Explain	CS	1
DM-Relational	3. Mapping conceptual schema to a relational schema 4. Physical database design: file and storage structures 5. Introduction to Functional dependency theory 6. Normalization theory a. Decomposition of a schema; lossless-join and dependency-preservation properties of a decomposition b. Normal forms (BCNF) c. Denormalization (for efficiency)	Develop	KA	3
DM-	1. SQL Query Formation	Develop	CS	2

Querying	a. Interactive SQL execution b. Programmatic execution of an SQL query			
DM-Querying	2. Relational Algebra 3. SQL a. Data definition including integrity and other constraints specification b. Update sublanguage	Develop	KA	4
DM-Processing	1. Page structures 2. Index structures a. B+ trees b. Hash indices: static and dynamic c. Index creation in SQL 3. File Structures a. Heap files b. Hash files 4. Algorithms for query operators a. External Sorting b. Selection c. Projection; with and without duplicate elimination d. Natural Joins: Nested loop, Sort-merge, Hash join e. Analysis of algorithm efficiency 5. Query transformations 6. Query optimization a. Access paths b. Query plan construction c. Selectivity estimation d. Index-only plans 7. Parallel Query Processing (e.g., parallel scan, parallel join, parallel aggregation)	Explain	KA	4
	8. Database tuning/performance a. Index selection b. Impact of indices on query performance c. Denormalization	Develop		
DM-Internals	1. DB Buffer Management 2. Transaction Processing a. Isolation Levels b. ACID	Explain	KA	4

	c. Serializability d. Distributed transactions 3. Concurrency Control: a. 2-Phase Locking b. Deadlocks handling strategies c. Quorum-based consistency models 4. Recovery Manager a. Relation with Buffer Manager			
DM-NoSQL	1. Why NoSQL? (e.g., Impedance mismatch between Application [CRUD] and RDBMS) 2. Key-Value and Document data model	Explain	KA	2
DM-Analytics	1. Exploratory data techniques (motivation, representation, descriptive statistics, visualizations) 2. Data science lifecycle – business understanding, data understanding, data preparation, modeling, evaluation, deployment, and user acceptance 3. Data mining and machine learning algorithms: e.g., classification, clustering, association, regression 4. Data acquisition and governance 5. Data security and privacy considerations 6. Data fairness and bias 7. Data visualization techniques and their use in data analytics 8. Entity Resolution	Explain	KA	3
DM-Security	1. Differences between data security and data privacy 2. Protecting data and database systems from attacks, including injection attacks such as SQL injection 3. Personally identifying information (PII) and its protection 4. Ethical considerations in ensuring the security and privacy of data	Explain	CS	1
DM-Security	5. Need for, and different approaches to securing data at rest, in transit, and during processing 6. Database auditing and its role in digital forensics 7. Data inferencing and preventing attacks	Explain	KA	2

	8. Laws and regulations governing data security and data privacy			
DM-SEP	1. Issues related to scale 2. Data privacy overall a. Privacy compliance by design 3. Data anonymity 4. Data ownership/custodianship 5. Intended and unintended applications of stored data	Explain	CS	
DM-SEP	6. Reliability of data 7. Provenance, data lineage, and metadata management 8. Data security	Explain	KA	

SEC: Security

KU	Topic	Skill Level	Core	Hours
SEC-Foundations	1. Developing a security mindset incorporating crosscutting concepts: confidentiality, integrity, availability, risk assessment, systems thinking, adversarial thinking, human-centered thinking 2. Basic concepts of authentication and authorization/access control 3. Vulnerabilities, threats, attack surfaces, and attack vectors 4. Denial of Service (DoS) and Distributed Denial of Service (DDoS) 5. Principles and practices of protection, e.g., least privilege, open design, fail-safe defaults, defense in depth, and zero trust; and how they can be implemented 6. Optimization considerations between security, privacy, performance, and other design goals 7. Impact of AI on security and privacy: using AI to bolster defenses as well as address increased adversarial capabilities due to AI	Develop	CS	1
SEC-Foundations	8. Access control models (e.g., discretionary, mandatory, role-based, and attribute-based) 9. Security controls	Develop	KA	5

	10. Concepts of trust and trustworthiness 11. Applications of a security mindset: web, cloud, and mobile devices 12. Protecting embedded and cyber-physical systems 13. Principles of usable security and human-centered computing 14. Security and trust in AI/machine learning systems, e.g., fit for purpose, ethical operating boundaries, authoritative knowledge sources, verified training data, repeatable system evaluation tests, system attestation, independent validation/certification; unintended consequences from: adverse effect 15. Security risks in building and operating AI/machine learning systems, e.g., algorithm bias, knowledge corpus bias, training corpus bias, copyright violation 16. Hardware considerations in security, e.g., principles of secure hardware, secure processor architectures, cryptographic acceleration, compartmentalization, software-hardware interaction			
SEC-SEP	1. Principles and practices of privacy 2. Societal impacts on breakdowns in security and privacy. 3. Applicability of laws and regulations on security and privacy 4. Professional ethical considerations when designing secure systems and maintaining privacy; ethical hacking	Develop	CS	1
SEC-SEP	5. Security by design 6. Privacy by design and privacy engineering 7. Security and privacy implications of malicious AI/machine learning actors, e.g., identifying deep fakes 8. Societal impacts of Internet of Things (IoT) devices and other emerging technologies on security and privacy	Develop	KA	2
SEC-Coding	1. Common vulnerabilities and weaknesses 2. SQL injection and other injection attacks 3. Cross-site scripting techniques and mitigations 4. Input validation and data sanitization 5. Type safety and type-safe languages	Develop	CS	2

	6. Buffer overflows, stack smashing, and integer overflows 7. Security issues due to race conditions			
SEC-Coding	8. Principles of noninterference and non-deducibility 9. Preventing information flow attacks 10. Offensive security techniques as a defense 11. AI-assisted malware detection techniques 12. Ransomware: creation, prevention, and mitigation. 13. Secure use of third-party components 14. Malware: varieties, creation, reverse engineering, and defense against them 15. Assurance: testing (including fuzzing and penetration testing), verification and validation 16. Static and dynamic analyses 17. Secure compilers and secure code generation	Develop	KA	5
SEC-Crypto	1. Differences between algorithmic, applied, and mathematical views of cryptography 2. Mathematical preliminaries: modular arithmetic, Euclidean algorithm, probabilistic independence, linear algebra basics, number theory, finite fields, complexity, asymptotic analysis. 3. Basic cryptography – symmetric key and public key cryptography 4. Basic cryptographic building blocks, including symmetric encryption, asymmetric encryption, hashing, and message authentication 5. Classical cryptosystems, such as shift, substitution, transposition ciphers, code books, machines 6. Kerckhoff's principle and use of vetted libraries 7. History and real-world applications, e.g., electronic cash, secure channels between clients and servers, secure electronic mail, entity authentication, device pairing, voting systems	Evaluate	CS	1
SEC-Crypto	8. Additional mathematics – primality and factoring; elliptic curve cryptography 9. Private-key cryptosystems – substitution-permutation networks, linear cryptanalysis, differential cryptanalysis, DES, AES 10. Public-key cryptosystems – Diffie-Hellman, RSA 11. Data integrity and authentication – hashing, digital signatures	Develop	KA	4

	12. Cryptographic protocols – challenge-response authentication, zero-knowledge protocols, commitment, oblivious transfer, secure two- or multi-party computation, hash functions, secret sharing, and applications 13. Attacker capabilities – chosen-message attack (for signatures), birthday attacks, side channel attacks, fault injection attacks 14. Quantum cryptography – Post Quantum/Quantum resistant cryptography 15. Blockchain and cryptocurrencies			
SEC-Engineering	1. Security engineering goals – building systems that remain dependable despite errors, accidents, or malicious adversaries 2. Privacy engineering goals – building systems that design, implement, and deploy privacy features and controls 3. Problem analysis and situational analysis to address system security 4. Engineering tradeoff analysis based on time, cost, risk tolerance, risk acceptance, return on investment, and so on	Develop	CS	1
SEC-Engineering	5. Security design and engineering, including functional requirements, security subsystems, information protection, security testing, security assessment, and evaluation 6. Security analysis, covering security requirements analysis; security controls analysis; threat analysis; and vulnerability analysis 7. Security attack domains and attack surfaces, e.g., communications and networking, hardware, physical, social engineering, software, and supply chain 8. Security attack modes, techniques, and tactics, e.g., authentication abuse; brute force; buffer manipulation; code injection; content insertion; denial of service; eavesdropping; function bypass; impersonation; integrity attack; interception; phishing; protocol analysis; privilege abuse; spoofing; and traffic injection 9. Attestation of software products with respect to their specification and adaptiveness 10. Design and development of cyber-physical systems	Develop	KA	8

	11. Considerations for trustworthy computing, e.g., tamper resistant packaging, trusted boot, trusted kernel, hardware root of trust, software signing and verification, hardware-based cryptography, virtualization, and containers			
SEC-Forensics	1. Basic principles and methodologies for digital forensics 2. System design for forensics 3. Forensics in different situations – operating systems, file systems, application forensics, web forensics, network forensics, mobile device forensics, use of database auditing 4. Attacks on forensics and preventing such attacks 5. Incident handling processes 6. Rules of evidence – general concepts and differences between jurisdictions 7. Legal issues – digital evidence protection and management, chains of custody, reporting, serving as an expert witness.	Develop	KA	6
SEC-Governance	1. Protecting critical assets from threats 2. Security governance – organizational objectives and general risk assessment 3. Security management – achieve and maintain appropriate levels of confidentiality, integrity, availability, accountability, authenticity, and reliability 4. Security policy – Organizational policies, issue-specific policies, system-specific policies 5. Approaches to identifying and mitigating risks to computing infrastructure 6. Data lifecycle management policies: data collection, backups, and retention; cloud storage and services; breach disclosure	Develop	KA	3

Applications Competency Area

The core topics listed in this section are typically covered in advanced, often elective courses. Effort should be made to include the CS Core topics in earlier required courses in the curriculum.

	Knowledge Area	**Knowledge Units**	**CS Core**	**KA Core**

AI	Artificial Intelligence	12	12	18
GIT	Graphics and Interactive Techniques	11	4	70
HCI	Human-Computer Interaction	6	8	16
SPD	Specialized Platform Development	8	4	68
	Total		28	N/A

AI: Artificial Intelligence

KU	Topic	Skill Level	Core	Hours
AI-Introduction	1. Overview of AI problems, examples of successful recent AI applications 2. Definitions of agents with examples (e.g., reactive, deliberative) 3. What is intelligent behavior? a. The Turing test and its flaws b. Multimodal input and output c. Simulation of intelligent behavior d. Rational vs non-rational reasoning	Explain	CS	2
	4. Problem characteristics a. Fully versus partially observable b. Single vs multi-agent c. Deterministic versus stochastic d. Static vs dynamic e. Discrete versus continuous 5. Nature of agents a. Autonomous, semi-autonomous, mixed-initiative autonomy b. Reflexive, goal-based, and utility-based c. Decision making under uncertainty and with incomplete information d. The importance of perception and environmental interactions e. Learning-based agents f. Embodied agents i. sensors, dynamics, effectors	Evaluate		

	6. AI Applications, growth, and Impact (economic, societal, ethics)	Explain		
AI-Introduction	7. Practice identifying problem characteristics in example environments 8. Additional depth on nature of agents with examples 9. Additional depth on AI Applications, growth, and Impact (economic, societal, ethics)	Evaluate	KA	1
AI-Search AL-Foundational	1. State space representation of a problem a. Specifying states, goals, and operators b. Factoring states into representations (hypothesis spaces) c. Problem solving by graph search i. e.g., Graphs as a space, and tree traversals as exploration of that space ii. Dynamic construction of the graph (not given upfront)	Explain	CS	5
	2. Uninformed graph search for problem solving a. Breadth-first search b. Depth-first search i. With iterative deepening c. Uniform cost search	Develop, Apply		
	3. Heuristic graph search for problem solving a. Heuristic construction and admissibility b. Hill-climbing c. Local minima and the search landscape i. Local vs global solutions d. Greedy best-first search e. A* search	Develop, Apply		
	4. Space and time complexities of graph search algorithms	Evaluate		
AI-Search	5. Bidirectional search 6. Beam search 7. Two-player adversarial games a. Minimax search b. Alpha-beta pruning i. Ply cutoff 8. Implementation of A* search 9. Constraint Satisfaction	Develop, Apply	KA	6

AI-KRR	1. Types of representations a. Symbolic, logical i. Creating a representation from a natural language problem statement b. Learned subsymbolic representations c. Graphical models (e.g., naive Bayes, Bayes net) 2. Review of probabilistic reasoning, Bayes theorem	Explain	CS	2
	3. Bayesian reasoning a. Bayesian inference	Apply		
AI-KRR	4. Random variables and probability distributions a. Axioms of probability b. Probabilistic inference c. Bayes' Rule (derivation) d. Bayesian inference (more complex examples) 5. Independence 6. Conditional Independence 7. Markov chains and Markov models 8. Utility and decision making	Apply	KA	2
AI-ML	1. Definition and examples of a broad variety of machine learning tasks a. Supervised learning i. Classification ii. Regression b. Reinforcement learning c. Unsupervised learning i. Clustering 2. Fundamental ideas: a. No free lunch theorem: no one learner can solve all problems; representational design decisions have consequences b. sources of error and undecidability in machine learning		CS	4
	3. A simple statistical-based supervised learning such as linear regression or decision trees a. Focus on how they work without going into mathematical or optimization details; enough to understand and use existing implementations correctly.	Apply, Develop, Evaluate		

	4. The overfitting problem / controlling solution complexity (regularization, pruning – intuition only) a. The bias (underfitting) – variance (overfitting) tradeoff 5. Working with Data a. Data preprocessing i. Importance and pitfalls of preprocessing choices b. Handling missing values (imputing, flag-as-missing) i. Implications of imputing vs flag-as-missing c. Encoding categorical variables, encoding real-valued data d. Normalization/standardization e. Emphasis on real data, not textbook examples 6. Representations a. Hypothesis spaces and complexity b. Simple basis feature expansion, such as squaring univariate features c. Learned feature representations 7. Machine learning evaluation a. Separation of train, validation, and test sets b. Performance metrics for classifiers c. Estimation of test performance on held-out data d. Tuning the parameters of a machine learning model with a validation set e. Importance of understanding what a model is doing, where its pitfalls/shortcomings are, and the implications of its decisions 8. Basic neural networks a. Fundamentals of understanding how neural networks work and their training process, without details of the calculations b. Basic introduction to generative neural networks (large language models, etc.)			
	9. Ethics for Machine Learning a. Focus on real data, real scenarios, and case studies b. Dataset/algorithmic/evaluation bias and unintended consequences	Explain, Evaluate		

AI-ML	10. Formulation of simple machine learning as an optimization problem, such as least squares linear regression or logistic regression a. Objective function b. Gradient descent c. Regularization to avoid overfitting (mathematical formulation) 11. Ensembles of models a. Simple weighted majority combination 12. Deep learning a. Deep feed-forward networks (intuition only, no mathematics) b. Convolutional neural networks (intuition only, no mathematics) c. Visualization of learned feature representations from deep nets d. Other architectures (generative NN, recurrent NN, transformers, etc.) 13. Performance evaluation a. Other metrics for classification (e.g., error, precision, recall) b. Performance metrics for regressors c. Confusion matrix d. Cross-validation i. Parameter tuning (grid/random search, via cross-validation) 14. Overview of reinforcement learning methods 15. Two or more applications of machine learning algorithms a. e.g., medicine and health, economics, vision, natural language, robotics, game play	Apply, Develop, Evaluate	KA	6
	16. Ethics for Machine Learning a. Continued focus on real data, real scenarios, and case studies b. Privacy c. Fairness d. Intellectual property e. Explainability	Explain, Evaluate		
AI-SEP	1. At least one application of AI to a specific problem and field, such as medicine, health, sustainability, social media, economics, education, robotics, etc. (at least one for the CS Core)	Explain, Evaluate	CS	3

	a. Formulating and evaluating a specific application as an AI problem i. How to deal with underspecified or ill-posed problems b. Data availability/scarcity and cleanliness i. Basic data cleaning and preprocessing ii. Data set bias c. Algorithmic bias d. Evaluation bias e. Assessment of societal implications of the application 2. Deployed deep generative models a. High-level overview of deep image generative models (e.g., as of 2023, DALL-E, Midjourney, Stable Diffusion), their uses, and their shortcomings/pitfalls. b. High-level overview of large language models (e.g., as of 2023, ChatGPT, Bard), their uses, and their shortcomings/pitfalls. 3. Overview of societal impact of AI a. Ethics b. Fairness c. Trust/explainability d. Privacy and usage of training data e. Human autonomy and oversight/regulations/legal requirements f. Sustainability			
AI-SEP	4. One or more additional applications of AI to a broad set of problems and diverse fields, such as medicine, health, sustainability, social media, economics, education, robotics, etc. (choose a different area from that chosen for the CS Core). a. Formulating and evaluating a specific application as an AI problem i. How to deal with underspecified or ill-posed problems b. Data availability/scarcity and cleanliness i. Basic data cleaning and preprocessing ii. Data set bias c. Algorithmic bias d. Evaluation bias	Explain, Evaluate	KA	3

	e. Assessment of societal implications of the application 5. Additional depth on deployed deep generative models a. Introduction to how deep image generative models work, (e.g., as of 2023, DALL-E, Midjourney, Stable Diffusion) including discussion of attention b. Introduction to how large language models work, (e.g., as of 2023, ChatGPT, Bard) including discussion of attention c. Idea of foundational models, how to use them, and the benefits/issues with training them from big data 6. Analysis and discussion of the societal impact of AI a. Ethics b. Fairness c. Trust/explainability d. Privacy and usage of training data e. Human autonomy and oversight/regulations/legal requirements f. Sustainability			

GIT: Graphics and Interactive Techniques

KU	Topic	Skill Level	Core	Hours
GIT-Fundamentals	1. Uses and potential risks 2. Graphic output 3. Human vision system 4. Standard image formats 5. Digitization of analog data 6. Color Models 7. Tradeoffs between storing image data and recomputing image data 8. Spatialization 9. Animation	Explain	CS	4
	10. Applied computer graphics 11. Display characteristics	Develop	KA	3
	1. Data visualization and information visualization	Explain	KA	6

GIT-Visualization	2. Visualization of: a. 2D/3D scalar fields b. Vector fields and flow data c. Time-varying data d. High-dimensional data e. Non-spatial data 3. Visualization techniques (e.g., color mapping, isosurfaces, dimension reduction, parallel coordinates, multi-variate, tree/graph-structured, text) 4. Direct volume data rendering (e.g., ray-casting, transfer functions, segmentation) 5. Common data formats (e.g., HDF, netCDF, geotiff, GeoJSON, shape files, raw binary, CSV, ASCII to parse) 6. Common visualization software and libraries (e.g., R, Processing, D3.js, GIS, Matlab, IDL, Python) 7. Perceptual and cognitive foundations that drive visual abstractions 8. Visualization design 9. Evaluation of visualization methods and applications 10. Visualization bias 11. Applications of visualization			
GIT-Rendering	1. Object and scene modeling 2. Camera and projection modeling 3. Light models and radiometry 4. Rendering	Explain	KA	6
GIT-Modeling	1. Basic geometric operations 2. Surface representation/model 3. Volumetric representation/model 4. Procedural representation/model 5. Multi-resolution modeling 6. Reconstruction	Explain	KA	6
GIT-Shading	1. Solutions and approximations to the rendering equation 2. Time (motion blur), lens position (focus), and continuous frequency (color) and their impact on rendering 3. Shadow mapping 4. Occlusion culling 5. Bidirectional Scattering Distribution function (BSDF) theory and microfacets	Explain	KA	6

	6. Subsurface scattering 7. Area light sources 8. Hierarchical depth buffering 9. Image-based rendering 10. Non-photorealistic rendering 11. GPU architecture 12. Human visual systems including adaptation to light, sensitivity to noise, and flicker fusion			
GIT-Animation	1. Principles of Animation 2. Types of animation 3. Key-frame animation 4. Forward and inverse kinematics 5. Skinning algorithms 6. Motion capture	Explain	KA	6
GIT-Simulation	1. Collision detection and response 2. Procedural animation using noise 3. Particle systems 4. Grid-based fluids 5. Heightfields 6. Rule-based systems (e.g., L-systems, space-colonizing systems, Game of Life)	Explain	KA	6
GIT-Immersion	1. Immersion levels (i.e., Virtual Reality (VR), Augmented Reality (AR), and Mixed Reality (MR)) 2. The definition and distinction of immersion and presence 3. 360 Video 4. Stereoscopic display 5. Viewer tracking 6. Time-critical rendering to achieve optimal motion to photon (MTP) latency a. Branching movies 7. Distributed VR, collaboration over computer network 8. Presence and factors that impact level of immersion 9. 3D interaction 10. Applications in medicine, simulation, training, and visualization 11. Safety in immersive applications	Explain	KA	6
GIT-Interaction	1. Event Driven Programming 2. Graphical User Interface (Single Channel) 3. Accessibility	Apply	KA	4

GIT-Image	1. Morphological operations 2. Color histograms 3. Image enhancement 4. Image restoration 5. Image coding 6. Connections to deep learning	Explain	KA	6
GIT-Physical	1. Interaction with the physical world 2. Connection to physical artifacts 3. Internet of Things	Explain	KA	6
GIT-SEP	1. Accessibility 2. Ethics/privacy 3. Intellectual Property law as it relates to computer graphics and interactive techniques 4. DEIA, current and past contributors to the field	Evaluate	CS KA	1 3

HCI: Human-Computer Interaction

KU	Topic	Skill Level	Core	Hours
HCI-User	1. User-centered design and evaluation methods a. "You are not the users" b. user needs-finding c. formative studies d. interviews e. surveys f. usability tests	Explain, Evaluate, Develop	CS	2
HCI-User	2. User-centered design and evaluation methods a. personas/persona spectrum b. user stories/storytelling and techniques for gathering stories c. empathy maps d. needs assessment (techniques for uncovering needs and gathering requirements – e.g., interviews, surveys, ethnographic and contextual enquiry) e. journey maps f. evaluating the design g. interfacing with stakeholders, as a team	Explain, Evaluate, Develop	KA	5

	h. risks associated with physical, distributed, hybrid and virtual teams 3. Physical and cognitive characteristics of the user a. physical capabilities that inform interaction design (e.g., color perception, ergonomics) b. cognitive models that inform interaction design (e.g., attention, perception and recognition, movement, memory) c. topics in social/behavioral psychology (e.g., cognitive biases, change blindness) 4. Designing for diverse user populations a. how differences (e.g., in race, ability, age, gender, culture, experience, and education) impact user experiences and needs b. internationalization, other cultures, and cross-cultural design c. designing for users from other cultures d. cross-cultural design e. challenges to effective design evaluation (e.g., sampling, generalization; disability and disabled experiences) f. universal design 5. Collaboration and communication a. understanding the user in a multi-user context b. synchronous group communication (e.g., chat rooms, conferencing, online games) c. asynchronous group communication (e.g., email, forums, social networks) d. social media, social computing, and social network analysis e. online collaboration f. social coordination and online communities g. avatars, characters, and virtual worlds			
HCI-Accountability	1. Design impact a. Sustainability b. Inclusivity c. Safety, security, and privacy d. Harm and disparate impact 2. Ethics in design methods and solutions a. the role of artificial intelligence b. responsibilities for considering stakeholder impact and human factors c. the role of design to meet user needs	Explain, Apply, Evaluate Develop	CS	2

	3. Requirements in design a. ownership responsibility b. legal frameworks and compliance requirements c. consideration beyond immediate user needs including via iterative reconstruction of problem analysis and “digital well-being” features			
HCI-Accountability	4. Value-sensitive design a. identify stakeholders b. determine and include diverse stakeholder values and value systems 5. Persuasion through design a. assessing persuasive content of a design b. persuasion as a design goal	Explain, Apply, Evaluate Develop	KA	2
HCI-Accessibility	1. Background a. societal and legal support for and obligations to people with disabilities b. accessible design benefits everyone 2. Techniques a. accessibility standards (e.g., Web Content Accessibility Guidelines) 3. Technologies a. features and products that enable accessibility and support inclusive development by designers and engineers 4. Inclusive Design Frameworks (IDFs) a. recognizing differences 5. Universal design	Explain, Apply, Evaluate Develop	CS	2
HCI-Accessibility	6. Background a. Demographics and populations b. International perspectives c. Attitudes towards people with disabilities 7. Techniques a. UX (user experience) design and research b. software engineering practices that enable inclusion and accessibility. 8. Technologies a. examples of accessibility-enabling features, such as conformance to screen readers 9. Inclusive Design Frameworks a. creating inclusive processes, such as participatory design; designing for larger impact	Explain, Apply, Evaluate Develop	KA	2

	b. designing for larger impact			
HCI-Evaluation	1. Methods for evaluation with users a. formative (e.g., needs-finding and exploratory analysis) and summative assessment (e.g., functionality and usability testing) b. elements to evaluate (e.g., utility, efficiency, learnability, user satisfaction, affective elements such as pleasure and engagement) c. understanding ethical approval requirements before engaging in user research	Explain, Apply, Evaluate, Develop	CS	1
HCI-Evaluation	2. Methods for evaluation with users a. qualitative methods (qualitative coding and thematic analysis) b. quantitative methods (statistical tests) c. mixed methods (e.g., observation, think-aloud, interview, survey, experiment) d. presentation requirements (e.g., reports, personas) e. user-centered testing f. heuristic evaluation g. challenges and shortcomings to effective evaluation (e.g., sampling, generalization) 3. Study planning a. how to set study goals b. hypothesis design c. approvals from Institutional Research Boards and ethics committees d. how to pre-register a study e. within-subjects vs between-subjects design 4. Implications and impacts of design with respect to a. the environment b. material c. society d. security e. privacy f. ethics g. broader impacts	Explain, Apply, Evaluate, Develop	KA	2
HCI-Design	1. Prototyping techniques and tools a. low-fidelity prototyping b. rapid prototyping c. throw-away prototyping	Explain, Apply, Evaluate, Develop	CS	1

	d. granularity of prototyping 2. Design patterns a. iterative design b. universal design c. interaction design (e.g., data-driven design, event-driven design 3. Design constraints a. platforms b. devices c. resources d. balance among usability, security and privacy			
HCI-Design	4. Design patterns and guidelines a. software architecture patterns b. cross-platform design c. synchronization 5. Design processes a. participatory design b. co-design c. double-diamond d. convergence and divergence 6. Interaction techniques a. input and output vectors (e.g., gesture, pose, touch, voice, force) b. graphical user interfaces c. controllers d. haptics e. hardware design f. error handling 7. Visual UI design a. color b. layout c. gestalt principles	Explain, Apply, Evaluate Develop	KA	5
HCI-SEP	1. Universal and user-centered design 2. Accountability 3. Accessibility and inclusive design 4. Evaluating the design 5. System design	Explain, Apply, Evaluate, Develop	CS	Shared with SEP
HCI-SEP	6. Participatory and inclusive design processes	Explain, Apply, Evaluate, Develop	KA	Shared with SEP

SPD: Specialized Platform Development

KU	Topic	Skill Level	Core	Hours
SPD-Common	1. Overview of development platforms (i.e., web, mobile, game, robotics, embedded, and interactive) a. Input/sensors/control devices/haptic devices b. Resource constraints i. Computational ii. Data storage iii. Memory iv. Communication c. Requirements – security, uptime availability, fault tolerance d. Output/actuators/haptic devices 2. Programming via platform-specific Application Programming Interface (API) vs traditional application construction 3. Overview of platform Languages (e.g., Python, Swift, Lua, Kotlin) 4. Programming under platform constraints and requirements (e.g., available development tools, development, security considerations) 5. Techniques for learning and mastering a platform-specific programming language	Apply	CS	3
SPD-Web	1. Web programming languages (e.g., HTML5, JavaScript, PHP, CSS) 2. Web platforms, frameworks, or meta-frameworks a. Cloud services b. API, Web Components 3. Software as a Service (SaaS). 4. Web standards such as document object model, accessibility 5. Security and Privacy Considerations	Apply	KA	5
SPD-Mobile	1. Development with a. Mobile programming languages b. Mobile programming environments	Apply	KA	3

	2. Mobile platform constraints a. User interface design b. Security 3. Access a. Accessing data through API b. Designing API endpoints for mobile apps – pitfalls and design considerations c. Network and the web interfaces			
SPD-Robot	1. Types of robotic platforms and devices 2. Sensors, embedded computation, and effectors (actuators) 3. Robot-specific languages and libraries 4. Robotic software architecture (e.g., using the Robot Operating System (ROS)) 5. Robotic platform constraints and design considerations 6. Interconnections with physical or simulated systems 7. Robotic Algorithms 8. Forward kinematics a. Inverse kinematics b. Dynamics c. Navigation and path planning d. Grasping and manipulation 9. Safety and interaction considerations	Apply	KA	4
SPD-Embedded	1. Introduction to the unique characteristics of embedded systems a. Real-time vs soft real-time and non-real-time systems b. Resource constraints, such as memory profiles and deadlines c. API for custom architectures d. GPU technology. e. Field Programmable Gate Arrays (FPGA). f. Cross-platform systems 2. Embedded Systems a. Microcontrollers b. Interrupts and feedback c. Interrupt handlers in high-level languages d. Hard and soft interrupts and trap-exits e. Interacting with hardware, actuators, and sensors	Apply	KA	4

	f. Energy efficiency g. Loosely timed coding and synchronization h. Software adapters 3. Embedded programming 4. Hard real-time systems vs soft real-time systems a. Timeliness b. Time synchronization/scheduling c. Prioritization d. Latency e. Compute jitter 5. Real-time resource management. 6. Memory management: a. Mapping programming construct (variable) to a memory location b. Shared memory c. Manual memory management d. Garbage collection e. Safety considerations and safety analysis. 7. Sensors and actuators 8. Analysis and verification 9. Application design			
SPD-Game	1. Historical and contemporary platforms for games a. Evolution of Game Platforms (e.g., Brown Box to Metaverse and beyond) Improvement in Computing Architectures (CPU and GPU); Platform Convergence and Mobility) b. Typical Game Platforms (e.g., Personal Computer; Home Console; Handheld Console; Arcade Machine; Interactive Television; Mobile Phone; Tablet; Integrated Head-Mounted Display; Immersive Installations and Simulators; Internet of Things enabled Devices; CAVE Systems; Web Browsers; Cloud-based Streaming Systems) c. Characteristics and Constraints of Different Game Platforms (e.g., Features (local storage, internetworking, peripherals); Run-time performance (GPU/CPU frequency, number of cores); Chipsets (physics processing units, vector co-processors);	Apply	KA	4

	Expansion Bandwidth (PCIe); Network throughput (Ethernet); Memory types and capacities (DDR/GDDR); Maximum stack depth; Power consumption; Thermal design; Endian) d. Typical Sensors, Controllers, and Actuators (e.g., distinctive control system designs – peripherals (mouse, keypad, joystick), game controllers, wearables, interactive surfaces; electronics and bespoke hardware; computer vision, inside-out tracking, and outside-in tracking; IoT-enabled electronics and I/O e. eSports Ecosystems (e.g., evolution of gameplay across platforms; games and eSports; game events such as LAN/arcade tournaments and international events such as the Olympic eSports Series; streamed media and spectatorship; multimedia technologies and broadcast management; professional play; data and machine learning for coaching and training) 2. Real-time Simulation and Rendering Systems a. CPU and GPU architectures: (e.g., Flynn's taxonomy; parallelization; instruction sets; standard components—graphics compute array, graphics memory controller, video graphics array basic input/output system; bus interface; power management unit; video processing unit; display interface) b. Pipelines for physical simulations and graphical rendering (e.g., tile-based, immediate-mode) c. Common Contexts for Algorithms, Data Structures, and Mathematical Functions (e.g., game loops; spatial partitioning, viewport culling, and level of detail; collision detection and resolution; physical simulation; behavior for intelligent agents; procedural content generation) d. Media representations (e.g., I/O, and computation techniques for virtual worlds: audio; music; sprites; models and textures;			

	text; dialogue; multimedia (e.g., olfaction, tactile) 3. Game Development Tools and Techniques: a. Programming Languages (e.g., C++; C#; Lua; Python; JavaScript). b. Shader Languages (e.g., HLSL, GLSL; ShaderGraph) c. Graphics Libraries and APIs (e.g., DirectX; SDL; OpenGL; Metal; Vulkan; WebGL) d. Common Development Tools and Environments (e.g., IDEs; Debuggers; Profilers; Version Control Systems including those handling binary assets; Development Kits and Production/Consumer Kits; Emulators) 4. Game Engines a. Open Game Engines (e.g., Unreal; Unity; Godot; CryEngine; Phyre; Source 2; Pygame and Ren'Py; Phaser; Twine; SpringRTS) b. Techniques (e.g., Ideation, Prototyping, Iterative Design and Implementation, Compiling Executable Builds, Development Operations and Quality Assurance – Play Testing and Technical Testing, Profiling; Optimization, Porting; Internationalization and Localization, Networking) 5. Game Design a. Vocabulary (e.g., game definitions; mechanics-dynamics-aesthetics model; industry terminology; experience design; models of experience and emotion) b. Design Thinking and User-Centered Experience Design (e.g., methods of designing games; iteration, incrementing, and the double-diamond; phases of pre- and post-production; quality assurance, including alpha and beta testing; stakeholder and customer involvement; community management) c. Genres (e.g., adventure; walking simulator; first-person shooter; real-time strategy;			

	multiplayer online battle arena (MOBA); role-playing game (rpg)) d. Audiences and Player Taxonomies (e.g., people who play games; diversity and broadening participation; pleasures, player types, and preferences; Bartle, yee). e. Proliferation of digital game technologies to domains beyond entertainment (e.g., Education and Training; Serious Games; Virtual Production; eSports; Gamification; Immersive Experience Design; Creative Industry Practice; Artistic Practice; Procedural Rhetoric)			

Crosscutting Core Topics

The core topics in Society, Ethics, and the Profession (SEP) and Mathematical and Statistical Foundations (MSF) may be covered across the curriculum or in dedicated courses and benefit all the competency areas.

	Knowledge Area	**Knowledge Units**	**CS Core**	**KA Core**
SEP	Society, Ethics, and the Profession	11	18	14
MSF	Mathematical and Statistical Foundations	5	55	145
	Total		73	159

SEP: Society, Ethics, and the Profession

KU	**Topic**	**Skill Level**	**Core**	**Hours**
SEP-Context	1. Social implications (e.g., political and cultural ideologies) in a hyper-networked world where the capabilities and impact of social media, artificial intelligence and computing in general are rapidly evolving 2. Impact of computing applications (e.g., social media, artificial intelligence applications) on	Evaluate Evaluate	CS	3

	individual well-being, and safety of all kinds (e.g., physical, emotional, economic) 3. Consequences of involving computing technologies, particularly artificial intelligence, biometric technologies, and algorithmic decision-making systems, in civic life (e.g., facial recognition technology, biometric tags, resource distribution algorithms, policing software) and how human agency and oversight is crucial 4. How deficits in diversity and accessibility in computing affect society and what steps can be taken to improve equity in computing	Evaluate Evaluate		
SEP-Context	5. Growth and control of the internet, data, computing, and artificial intelligence 6. Often referred to as the digital divide, differences in access to digital technology resources and its resulting ramifications for gender, class, ethnicity, geography, and/or developing countries, including consideration of responsibility to those who might be less wealthy, under threat, or who would struggle to have their voices heard. 7. Accessibility issues, including legal requirements such as Web Content Accessibility Guidelines (www.w3.org/TR/WCAG21) 8. Context-aware computing	Explain Evaluate Explain Explain	KA	2
SEP-Ethical-Analysis	1. Avoiding fallacies and misrepresentation in argumentation 2. Ethical theories and decision-making (philosophical and social frameworks) 3. Recognition of the role culture plays in our understanding, adoption, design, and use of computing technology 4. Why ethics is important in computing, and how ethics is similar to, and different from, laws and social norms	Apply Apply Evaluate Explain	CS	2
SEP-Ethical-Analysis	5. Professional checklists 6. Evaluation rubrics 7. Stakeholder analysis 8. Standpoint theory 9. Introduction to ethical frameworks (e.g., consequentialism such as utilitarianism, non-	Develop Develop Develop Apply Explain	KA	1

	consequentialism such as duty, rights, or justice, agent-centered such as virtue or feminism, contractarianism, ethics of care) and their use for analyzing an ethical dilemma			
SEP-Professional-Ethics	1. Community values and the laws by which we live 2. The nature of being a professional including care, attention, discipline, fiduciary responsibility, and mentoring 3. Keeping up to date as a computing professional in terms of familiarity, tools, skills, legal and professional frameworks as well as the ability and responsibility to self-assess and progress in the computing field 4. Professional certification, codes of ethics, conduct, and practice, such as the ACM, IEEE, AAAI, and other international societies 5. Accountability, responsibility, and liability (e.g., software correctness, reliability and safety, warranty, negligence, strict liability, ethical approaches to security vulnerability disclosures) including whether a product/service should be built, not just doing so because it is technically possible. 6. Introduction to theories describing the human creation and use of technology including instrumentalism, sociology of technological systems, disability justice, neutrality thesis, pragmatism, and decolonial models, including developing and using technology to right wrongs and do good 7. Strategies for recognizing and reporting designs, systems, software, and professional conduct (or their outcomes) that may violate law or professional codes of ethics	Evaluate Apply Evaluate Evaluate Apply Explain Apply	CS	2
SEP-Professional-Ethics	8. The role of the computing professional and professional societies in public policy 9. Maintaining awareness of consequences 10. Ethical dissent and whistleblowing 11. The relationship between regional culture and ethical dilemmas 12. Dealing with harassment and discrimination 13. Forms of professional credentialing	Explain Apply Explain Evaluate Explain Explain	KA	2

	14. Ergonomics and healthy computing environments 15. Time-to-market and cost considerations versus quality professional standards	Explain Explain		
SEP-IP	1. Intellectual property rights 2. Intangible digital intellectual property (IDIP) 3. Legal foundations for intellectual property protection 4. Common software licenses (e.g., MIT, GPL and its variants, Apache, Mozilla, Creative Commons) 5. Plagiarism and authorship	Explain Explain Evaluate Evaluate Explain	CS	1
SEP-IP	6. Philosophical foundations of intellectual property 7. Forms of intellectual property (e.g., copyrights, patents, trade secrets, trademarks) and the rights they protect 8. Limitations on copyright protections, including fair use and the first sale doctrine 9. Intellectual property laws and treaties that impact the enforcement of copyrights 10. Software piracy and technical methods for enforcing intellectual property rights, such as digital rights management and closed source software as a trade secret 11. Moral and legal foundations of the open-source movement 12. Systems that use others' data (e.g., large language models)	Explain Explain Explain Explain Explain Explain	KA	1
SEP-Privacy	1. Privacy implications of widespread data collection including but not limited to transactional databases, data warehouses, surveillance systems, cloud computing, and artificial intelligence 2. Conceptions of anonymity, pseudonymity, and identity 3. Technology-based solutions for privacy protection (e.g., end-to-end encryption and differential privacy) 4. Civil liberties, privacy rights, and cultural differences	Explain Evaluate Evaluate Explain	CS	2
SEP-Privacy	5. Philosophical and legal conceptions of the nature of privacy including the right to privacy	Explain	KA	1

	6. Legal foundations of privacy protection in relevant jurisdictions (e.g., GDPR in the EU) 7. Privacy legislation in areas of practice (e.g., HIPAA in the US, AI Act in the EU) 8. Basic Principles of human-subjects research and principles beyond what the law requires (e.g., Belmont Report, UN Universal Declaration on Human Rights and how this relates to technology) 9. Freedom of expression and its limitations 10. User-generated content, content moderation, and liability	Explain Evaluate Explain Evaluate Explain		
SEP-Communication	1. Oral, written, and electronic team and group communication 2. Technical communication materials (e.g., source code and documentation, tutorials, reference materials, API documentation) 3. Communicating with different stakeholders such as customers, leadership, or the public 4. Team collaboration (including tools) and conflict resolution 5. Accessibility and inclusivity requirements for addressing professional audiences 6. Cultural competence in communication including considering the impact of difference in natural language	Apply Develop Apply Apply Apply Develop	CS	2
SEP-Communication	7. Tradeoffs in competing factors that affect communication channels and choices 8. Communicating to solve problems or make recommendations in the workplace, such as raising ethical concerns or addressing accessibility issues	Evaluate Apply	KA	1
SEP-Sustainability	1. Environmental, social, and cultural impacts of implementation decisions (e.g., sustainability goals, algorithmic bias/outcomes, economic viability, and resource consumption) 2. Local/regional/global social and environmental impacts of computing systems and their use (e.g., carbon footprints, resource usage, e-waste) in hardware (e.g., e-waste, data centers, rare element and resource utilization, recycling) and software (e.g., cloud-based services, blockchain,	Evaluate Evaluate	CS	1

	AI model training and use), not neglecting the impact of everyday use such as hardware (cheap hardware replaced frequently) and software (web-browsing, email, and other services with hidden/remote computational demands) 3. Guidelines for sustainable design standards	 Develop		
SEP-Sustainability	4. Systemic effects of complex computing technologies and phenomena (e.g., generative AI, data centers, social media, offshoring, remote work) 5. Pervasive computing – information processing that has been integrated into everyday objects and activities, such as smart energy systems, social networking, and feedback systems to promote sustainable behavior, transportation, environmental monitoring, citizen science and activism 6. How the sustainability of software systems is interdependent with social systems, including the knowledge and skills of its users, organizational processes and policies, and its societal context (e.g., market forces, government policies)	Evaluate Evaluate Explain	KA	1
SEP-History	1. The history of computing: hardware, software, and human/organizational 2. The role of history in the present including within different social contexts, and the relevance of this history on the future	Explain Evaluate	CS	1
SEP-History	3. Age I (Pre-digital) – Ancient analog computing (Stonehenge, Antikythera mechanism, Salisbury Cathedral clock, etc.), human-calculated number tables, Euclid, Lovelace, Babbage, Gödel, Church, Turing, pre-electronic (electro-mechanical and mechanical) hardware 4. Age II (Early modern computing) – ENIAC, UNIVAC, Bombes (Bletchley Park and codebreakers), computer companies (e.g., IBM), mainframes, etc. 5. Age III (PC era) – PCs, modern computer hardware and software, Moore's Law 6. Age IV (Internet) – Networking, internet architecture, browsers and their evolution,	Explain Explain Explain Explain	KA	1

	standards, born-on-the-internet companies, and services (e.g., Google, Amazon, Microsoft, etc.), distributed computing 7. Age V (Mobile & Cloud) – Mobile computing and smartphones, cloud computing and models thereof (e.g., SaaS), remote servers, security and privacy, social media 8. Age VI (AI) – Decision making systems, recommender systems, generative AI and other machine learning driven tools and technologies	Explain Explain		
SEP-Economies	1. Economic models – regulated and unregulated, monopolies, network effects, and open market; knowledge and attention economies 2. Pricing and deployment strategies – planned obsolescence, subscriptions, freemium, software licensing, open-source, free software, adware 3. Impacts of differences in access to computing resources, and the effect of skilled labor supply and demand on the quality of computing products 4. Automation, AI, and their effects on job markets, developers, and users 5. Ethical concerns surrounding the attention economy and other economies of computing (e.g., informed consent, data collection, use of verbose legalese in user agreements)	Explain Explain Evaluate Explain Evaluate	KA	1
SEP-Security	1. Computer crimes, legal redress for computer criminals and impact on victims and society 2. Social engineering, computing-enabled fraud, identity theft and recovery from these 3. Cyber terrorism, criminal hacking, and hacktivism 4. Malware, viruses, worms 5. Attacks on critical infrastructure such as electrical grids and pipelines 6. Non-technical fundamentals of security (e.g., human engineering, policy, confidentiality)	Explain Explain Explain Explain Explain Explain	CS	2
SEP-Security	7. Benefits and challenges of existing and proposed computer crime laws 8. Security policies and the challenges of change and compliance 9. Responsibility for security throughout the computing life cycle	Evaluate Explain Explain	KA	1

	10. International and local laws and how they intersect	Explain		
SEP-DEIA	1. How identity impacts and is impacted by computing technologies and environments (academic and professional) 2. The benefits of diverse development teams and the impacts of teams that are not diverse 3. Inclusive language and charged terminology, and why their use matters 4. Inclusive behaviors and why they matter 5. Designing and developing technology with accessibility in mind 6. How computing professionals can influence and impact diversity, equity, inclusion, and accessibility, including but not only through the software they create	Explain Explain Apply Explain Explain Explain	CS	2
SEP-DEIA	7. Experts and their practices that reflect the identities of the classroom and the world through practical DEIA principles 8. Historic marginalization due to systemic social mechanisms, technological supremacy and global infrastructure challenges to diversity, equity, inclusion, and accessibility 9. Cross-cultural differences in, and needs for, diversity, equity, inclusion, and accessibility	Evaluate Explain Explain	KA	2

MSF: Mathematical and Statistical Foundations

KU	Topic	Skill Level	Core	Hours
MSF-Discrete	1. Sets, relations, functions, cardinality 2. Recursive mathematical definitions 3. Proof techniques (induction, proof by contradiction) 4. Permutations, combinations, counting, pigeonhole principle 5. Modular arithmetic 6. Logic: truth tables, connectives (operators), inference rules, formulas, normal forms, simple predicate logic 7. Graphs: basic definitions 8. Order notation	Apply, Develop, Explain	CS/KA	29-40

MSF-Probability	1. Basic notions: sample spaces, events, probability, conditional probability, Bayes' rule 2. Discrete random variables and distributions 3. Continuous random variables and distributions 4. Expectation, variance, law of large numbers, central limit theorem 5. Conditional distributions and expectation 6. Applications to computing, the difference between probability and statistics (as subjects)	CS-Core: Apply KA-Core: Apply, Develop, Explain	CS/KA	11-40
MSF-Statistics	1. Basic definitions and concepts: populations, samples, measures of central tendency, variance 2. Univariate data: point estimation, confidence intervals	Develop	CS	10
MSF-Statistics	3. Multivariate data – estimation, correlation, regression 4. Data transformation – dimension reduction, smoothing 5. Statistical models and algorithms 6. Hypothesis testing	Apply, Explain	KA	30
MSF-Linear	1. Vectors – definitions, vector operations, geometric interpretation, angles; Matrices – definition, matrix operations, meaning of Ax=b	Develop	CS	5
MSF-Linear	2. Matrices, matrix-vector equation, geometric interpretation, geometric transformations with matrices 3. Solving equations, row-reduction 4. Linear independence, span, basis 5. Orthogonality, projection, least-squares, orthogonal bases 6. Linear combinations of polynomials, Bezier curves 7. Eigenvectors and eigenvalues 8. Applications to computer science – PCA, SVD, page-rank, graphics	Apply, Explain	KA	35
MSF-Calculus	1. Sequences, series, limits 2. Single-variable derivatives – definition, computation rules (chain rule, etc.), derivatives of important functions, applications 3. Single-variable integration – definition, computation rules, integrals of important functions, fundamental theorem of calculus, definite vs indefinite, applications (including in probability) 4. Parametric and polar representations	Apply, Develop	KA	40

	5. Taylor series 6. Multivariate calculus – partial derivatives, gradient, chain-rule, vector valued functions, 7. Optimization – convexity, global vs local minima, gradient descent, constrained optimization, and Lagrange multipliers 8. ODEs – definition, Euler method, applications to simulation, Monte Carlo integration 9. CS applications – gradient descent for machine learning, forward and inverse kinematics, applications of calculus to probability			